

Kapture Finance Collections Voicebot

High-Level Design (HLD)

Project: Maya — AI Collections Voice Assistant

Platform: Vapi

Backend: Node.js + Express.js

Backend Deployment: Render

Demo Frontend: Vercel

Speech-to-Text: Deepgram Nova-2

Text-to-Speech: Vapi-supported TTS provider

LLM: OpenAI model through Vapi

1. Project Overview

Maya is an AI-powered outbound collections voice assistant designed for Kapture Finance.

The primary purpose of Maya is to contact customers regarding overdue loan EMIs, verify the customer's identity before disclosing sensitive account information, understand the customer's payment intent, record a promise to pay, provide payment assistance, record dispositions, and escalate cases that require human intervention.

The solution was implemented as a working voicebot using Vapi for voice interaction and conversation orchestration, with a Node.js/Express mock backend deployed on Render for customer verification, account information, session state management, payment-related actions, disposition logging, and escalation.

A Vercel-hosted web page provides a simple interface through which an evaluator can start a conversation with Maya.

2. Objectives

The main objectives of the system are:

- Authenticate the customer before revealing sensitive financial information.
- Provide a natural voice-based collections conversation.
- Retrieve overdue account information after successful verification.
- Understand customer payment intent.
- Record a Promise to Pay (PTP).
- Support payment-link requests.
- Handle collection edge cases such as already-paid claims.
- Record conversation dispositions.

- Escalate appropriate cases to a human agent.
- Maintain controlled conversation states.
- Prevent protected actions before authentication.
- Provide call logs and tool execution information for monitoring and debugging.

3. High-Level Architecture

The system consists of the following major components:

1. Customer / Caller
2. Vapi Voice Assistant
3. Speech-to-Text
4. LLM / Conversation Orchestration
5. Text-to-Speech
6. Vapi Tool Calls
7. Express.js Mock Backend
8. Customer Data and Session State
9. Vapi Call Logs
10. Vercel Demo Frontend

Architecture Flow

Customer → Vapi → STT → LLM / Maya → Tool API → Render Backend → Tool Result → Maya → TTS → Customer

The Vapi assistant handles the conversation and determines when a backend function should be called.

The Render backend validates requests, checks authentication and conversation state, performs the corresponding mock operation, and returns a structured response to Vapi.

4. System Components

4.1 Vapi Voice Assistant

Vapi is responsible for:

- Handling the voice conversation.
- Receiving customer speech.
- Converting speech to text.

- Passing the conversation to the configured LLM.
- Following the Maya system prompt.
- Calling backend tools when required.
- Receiving tool results.
- Generating the next conversational response.
- Converting the response to speech.
- Managing the voice call.

Maya is configured as a collections assistant with a controlled conversational flow.

4.2 Speech-to-Text

Deepgram Nova-2 is used for speech recognition.

The customer's voice is converted into text so that Maya can understand:

- Verification information
- Payment intent
- PTP dates
- Already-paid claims
- Requests for payment links
- Requests for human assistance
- Other conversational responses

4.3 LLM / Conversation Orchestration

The LLM processes the conversation according to the Maya system prompt.

The assistant determines:

- What the customer is asking.
- Whether authentication has been completed.
- Whether account information can be disclosed.
- Which backend tool should be called.
- How to respond to the customer.
- Whether the conversation should continue, end, or be escalated.

The LLM does not directly modify customer data. Protected actions are performed through backend tools.

4.4 Express.js Backend

The backend is implemented using Node.js and Express.js.

The backend is deployed on Render and exposes a webhook endpoint:

POST /webhook

The endpoint receives tool requests from Vapi and returns structured JSON responses.

The backend contains a mock customer record for the assignment and maintains an in-memory session for the active account.

5. Customer Data

The demo uses a mock customer record containing information such as:

- Account ID
- Customer name
- Verification code
- EMI amount
- Due date
- Number of overdue days
- Registered phone number

Example demo customer:

Customer: Rahul Sharma

Account: ACC-88392

EMI: ₹8,499

Days overdue: 12 days

The customer information is used only for demonstrating the assignment workflow.

In a production system, this information would be retrieved from a secure customer/loan management system rather than hard-coded data.

6. Authentication Flow

Authentication is a critical part of the system.

Maya must not disclose the customer's overdue amount or loan information before successful verification.

The flow is:

1. Maya identifies herself and asks whether the intended customer is available.

2. Maya requests the configured verification information.
3. Maya calls `verify_customer`.
4. The backend checks the account ID and verification information.
5. If the information is correct, the session is marked as authenticated.
6. Only after successful authentication can protected account information be accessed.

If verification fails, Maya must not disclose sensitive financial information.

7. Conversation State Machine

The backend maintains a session state to control the conversation.

Main states

AUTHENTICATED



NEGOTIATION



PTP_COLLECTED



CALL_ENDED

An escalation path is also available:

AUTHENTICATED / NEGOTIATION / PTP_COLLECTED



ESCALATED

7.1 AUTHENTICATED

The customer has successfully completed verification.

Allowed actions include:

- Retrieving account details.
- Starting negotiation.
- Processing appropriate payment-related actions.

7.2 NEGOTIATION

The customer is discussing payment or deciding how and when to make the payment.

Maya can collect the customer's payment intention and proceed with the appropriate action.

7.3 PTP_COLLECTED

The customer has made a promise to pay.

The backend records the PTP state and allows the appropriate disposition/end-call flow.

7.4 ESCALATED

The conversation requires human intervention.

Examples include:

- Customer dispute.
- Hardship case.
- Hostile interaction.
- Unsupported request.
- Customer explicitly requesting a human agent.

The backend generates a ticket ID for the escalation.

7.5 CALL_ENDED

The routine conversation has been completed.

The backend prevents further routine escalation after the call has already been ended.

8. Backend Tools

8.1 verify_customer

Purpose

Verifies the customer's identity.

Inputs

- account_id
- verification_code

Output

Returns whether the customer was successfully verified.

On successful verification, the session state becomes:

AUTHENTICATED

8.2 get_account_details

Purpose

Retrieves the customer's overdue account information.

Inputs

- account_id

Security

The backend checks whether the customer has already been authenticated.

If the session is not authenticated, the backend returns an authorization error.

8.3 start_negotiation

Purpose

Moves an authenticated customer into the payment negotiation state.

State transition

AUTHENTICATED → NEGOTIATION

The backend rejects the request if the customer is not authenticated or is already in an invalid state.

8.4 log_promise_to_pay

Purpose

Records the customer's commitment to make a payment.

Inputs

- Account ID
- Promise date
- Payment information where applicable
- Notes where applicable

State transition

AUTHENTICATED / NEGOTIATION → PTP_COLLECTED

8.5 send_payment_link

Purpose

Handles a customer request for a payment link.

Input

- Account ID
- Channel

The current assignment implementation uses a mock response rather than sending a real financial transaction.

8.6 mark_disposition

Purpose

Records the final outcome of an interaction.

Examples include:

- Already paid
- Dispute
- Customer unavailable
- Other collection outcome

The backend records the status and optional notes.

8.7 escalate_to_agent

Purpose

Transfers cases requiring human intervention.

Inputs

- Account ID
- Reason
- Notes

The backend generates a mock ticket ID and changes the session state to:

ESCALATED

8.8 end_call

Purpose

Marks the completed conversation as ended.

State transition

PTP_COLLECTED → CALL_ENDED

9. Core Successful Conversation Flow

A successful PTP conversation follows this flow:

1. Maya calls the customer.
2. Customer confirms identity.
3. Maya requests verification information.
4. Customer provides the correct verification information.
5. verify_customer executes successfully.
6. Maya discloses the overdue EMI information.
7. Maya asks whether the customer can make the payment.
8. Customer provides a future payment date.
9. Maya confirms the promise.
10. log_promise_to_pay executes successfully.
11. The PTP is recorded.
12. Maya closes the conversation politely.
13. The call ends.

10. Already-Paid Edge Case

The system also handles the case where a customer says that the EMI has already been paid.

Example:

Customer: "Actually, I already paid this EMI."

Maya should not argue with the customer or ask them to make the same payment again.

Instead:

1. Acknowledge the customer's statement.

2. Ask when and through which payment method the payment was made, where appropriate.
3. Record the appropriate disposition using mark_disposition.
4. Provide a polite response.
5. End the call.

This behavior was tested successfully during the assignment demonstration.

11. Other Edge Cases

The conversation design considers the following cases:

Wrong Person

Maya must not reveal loan or overdue information if the person answering is not the intended customer.

Verification Failure

Maya must not disclose sensitive information when authentication fails.

Customer Dispute

Maya should not argue with the customer. The case should be recorded and escalated when required.

Customer Hardship

If the customer cannot make the payment because of financial difficulty, Maya should respond empathetically and follow the configured escalation or assistance process.

Human Agent Request

If the customer asks to speak with a human, Maya should use the escalation flow.

Do-Not-Call Request

The customer's request not to receive further calls should be respected and recorded according to the applicable business process.

Hostile Customer

Maya should remain calm and avoid confrontation. The conversation can be escalated or ended appropriately.

12. Security and Guardrails

The following safeguards are implemented or included in the conversation design:

- Authentication is required before accessing protected account information.

- Backend state is checked before protected operations.
- Debt information is not disclosed before successful verification.
- Tool results are treated as the source of truth for backend actions.
- Maya must not claim that an operation succeeded when the backend reports failure.
- The assistant should not invent account balances, dates, payment status, or policies.
- Sensitive information should not be unnecessarily repeated.
- Production credentials should be stored using secure environment variables.
- Human escalation is available for cases requiring manual handling.

13. Deployment Architecture

Vapi

Vapi hosts the voice assistant and handles the real-time voice interaction.

Render

The Express.js mock backend is deployed on Render.

Vapi sends tool requests to the Render webhook endpoint.

Vercel

The frontend demo is deployed on Vercel.

The frontend provides the evaluator with a simple way to access and start the Maya voice assistant.

GitHub

The source code, README, HLD and supporting documentation are maintained in the GitHub repository.

14. Observability and Debugging

Vapi call logs provide visibility into:

- Conversation transcript.
- Tool execution.
- Tool success/failure.
- Call duration.
- Customer and assistant audio.

- Conversation sequence.

The Render backend logs incoming webhook requests and request payloads.

This makes it possible to trace a tool call from:

Customer speech → Maya → Tool → Backend → Tool response → Maya response

Tool failures can therefore be identified and debugged separately from conversational issues.

15. Current Implementation Limitations

This project is a take-home assignment prototype and therefore uses mock services.

Current limitations include:

- Customer data is stored in a mock in-memory object.
- Session state is stored in memory.
- Payment-link generation is mocked.
- No real payment transaction is performed.
- The backend is not connected to a production loan management system.
- Production-grade authentication between services would be required.
- A production deployment would require persistent storage.
- Compliance rules would need to be configured according to the applicable jurisdiction and client policy.

16. Production Improvements

If this system were moved to production, the following improvements would be recommended:

1. Replace mock customer data with a secure loan/customer API.
2. Replace in-memory sessions with a persistent datastore such as Redis or a database.
3. Add authentication and authorization between Vapi and backend services.
4. Implement secure payment-link generation.
5. Add structured logging and monitoring.
6. Add rate limiting and request validation.
7. Add automated conversation tests.

8. Add configurable compliance rules.
9. Add persistent call and disposition records.
10. Add monitoring dashboards for PTP rate, escalation rate, tool failures, call duration and containment.
11. Add human-agent integration for escalated cases.
12. Add stronger protection for personally identifiable and financial information.

17. Demo Scenarios

The working demonstration covers the following scenarios:

Scenario 1 — Successful Promise to Pay

- Customer verification succeeds.
- Overdue EMI information is disclosed.
- Customer says they will pay tomorrow.
- PTP tool executes successfully.
- Maya confirms the commitment and closes the call.

Scenario 2 — Already Paid

- Customer verification succeeds.
- Customer states that the EMI has already been paid.
- Maya asks for payment information.
- `mark_disposition` executes successfully.
- Maya closes the conversation without asking the customer to pay again.

These scenarios demonstrate authentication, state management, tool calling, edge-case handling and appropriate conversation termination.

18. Conclusion

The Maya Collections Voice Assistant demonstrates an end-to-end voice AI collections workflow using Vapi, an Express.js backend and a simple web frontend.

The architecture separates voice interaction and conversational reasoning from backend business logic. Authentication and state validation are enforced by the backend, while Vapi manages the real-time conversational experience.

The prototype demonstrates the core collections workflow, successful Promise-to-Pay handling, an already-paid edge case, backend tool execution, escalation capability, call logging and a publicly accessible demo interface. The design can be extended into a production system by replacing the mock services with secure customer, payment, CRM and agent systems while adding persistent state, authentication, monitoring and jurisdiction-specific compliance controls.